

Observability in MOC 2.0

Introduce observability, show how you can access telemetry in MOC 2.0, and explain what kind of support you can expect from the MOC team.

Audience: Open Accelerator startups

Agenda

- 1 What is observability?
- 2 Which questions does your role ask?
- 3 How can I access the data?
- 4 Three layers of visibility
- 5 Matching questions to signals
- 6 Observability, inventory and billing
- 7 From telemetry to invoice
- 8 Purpose before tools
- 9 What we can help you with



Observability makes system behavior understandable

Something is slow. Something failed. Nobody can say why. Observability is how you answer that with data instead of guessing.

It helps us understand what is happening inside a system by looking at the signals it produces.

MOC 2.0 is the Mass Open Cloud platform you are running on. It is built on OpenShift and provides the telemetry we talk about in this session.

The three classic signals

- **Metrics** show how much, how fast and how often. Request rates, memory use, error counts.
- **Logs** explain what happened, as text written by your application and the platform.
- **Traces** show where time was spent, following one request across services.

Additional signals

- **Events** capture relevant state changes, for example a pod being rescheduled or an image failing to pull.
- **Continuous profiling** identifies expensive code paths inside a running process.
- **AI telemetry** adds signals specific to model serving: time to first token, token throughput, queue time and GPU utilization.

Not every additional signal is available everywhere. Ask us what is enabled for your project.

Key message Observability turns system activity into information we can actually investigate and act on.



Your role decides which questions you ask

In a small team, one person often wears all three hats. The roles below describe the questions, not job titles.

Developers ask about their own code

- Inspect their applications and pods
- Troubleshoot failures
- Understand performance and dependencies

Operations and administrators ask about the platform

- Monitor platform health and capacity
- Detect abnormal behavior
- Define alerts and investigate incidents

AI and RHOAI users ask about models

RHOAI = Red Hat OpenShift AI, the platform layer for model training and serving.

- Observe model serving workloads
- Analyze latency, throughput and GPU usage
- Monitor workbenches (interactive notebook environments), pipelines and inference services

Key message Which view is useful depends on your role, your permissions, and what you're actually responsible for.



Start in the console, grow into specialized tools

OpenShift web console

View workloads, resource usage, events, logs and available metrics right in the browser. No setup needed, start here.

Command line

Use `oc` for quick inspection and troubleshooting:

```
oc get pods
oc describe pod <pod>
oc logs <pod>
oc adm top pods    # needs elevated permissions, ask us if it is denied
```

Dashboards

Use **Grafana** or **Perses** to build visualizations, follow trends over time, and share one operational view across the team.

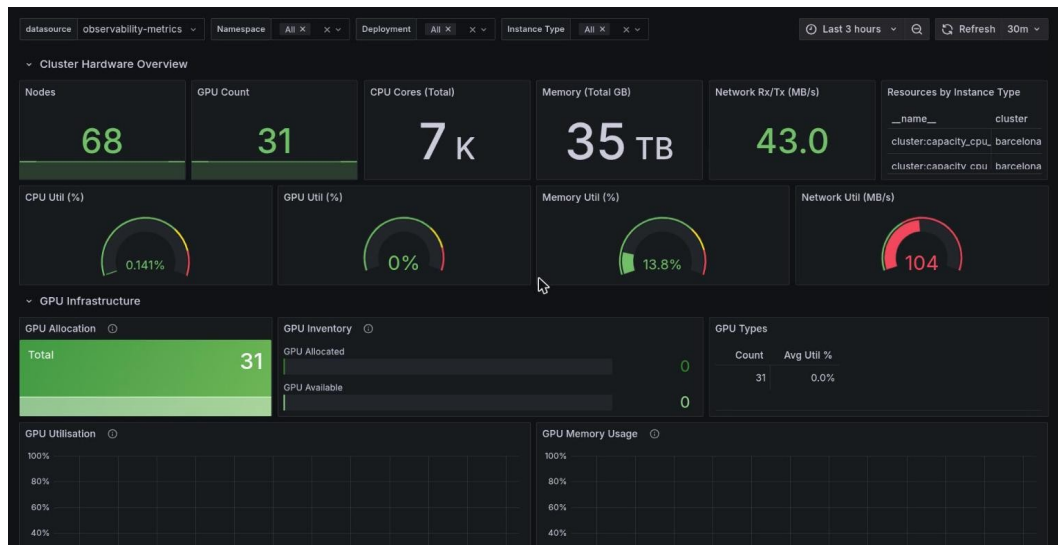
APIs and data sources

Query telemetry from external tools, or wire it into your own dashboards and applications.

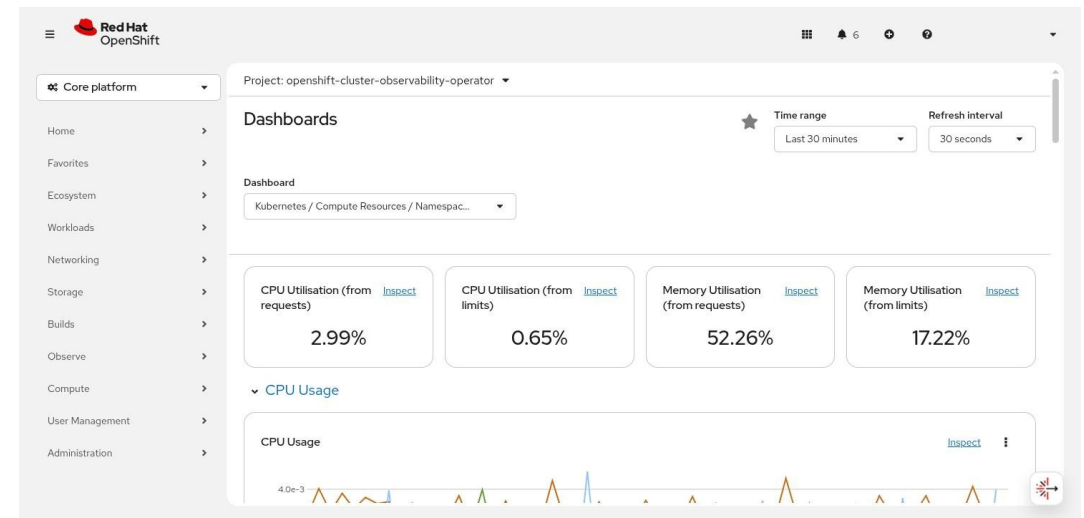
Key message Start in the web console and move to specialized tools once your needs actually grow past it.



Metrics: two views of the same cluster



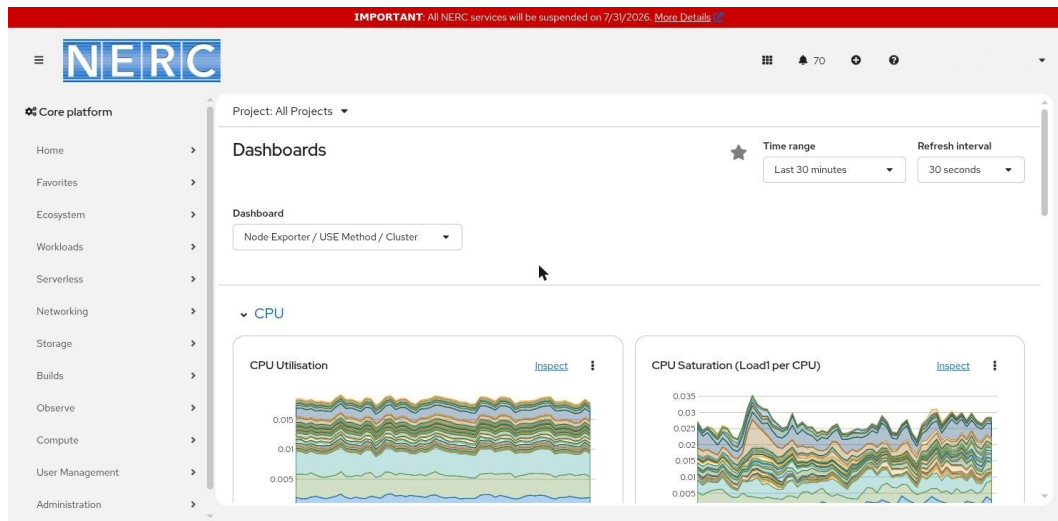
Grafana: nodes, GPUs, cores, running pods



OpenShift console: one namespace, requests against limits



Metrics: how busy, and how much is claimed



Console: CPU load across every node

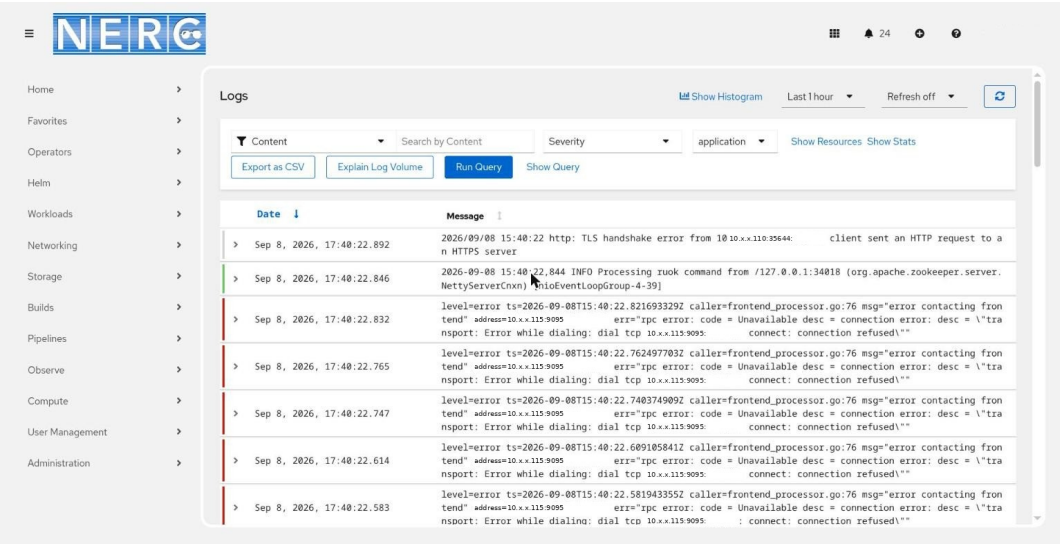
```
3 • How much of the node is claimed?
$ oc describe node <node>

node: moc-s3
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests          Limits
-----
cpu                 2917m (1%)        6410m (3%)
memory              8800Mi (0%)       10328Mi (0%)
ephemeral-storage   0 (0%)            0 (0%)
hugepages-1Gi       0 (0%)            0 (0%)
hugepages-2Mi       0 (0%)            0 (0%)
Events:             <none>
```

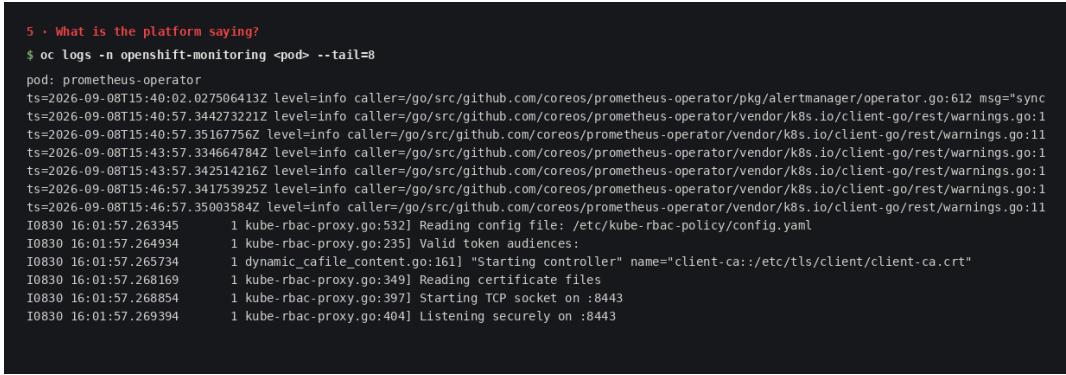
Terminal: what one node has been promised



Logs: two ways in



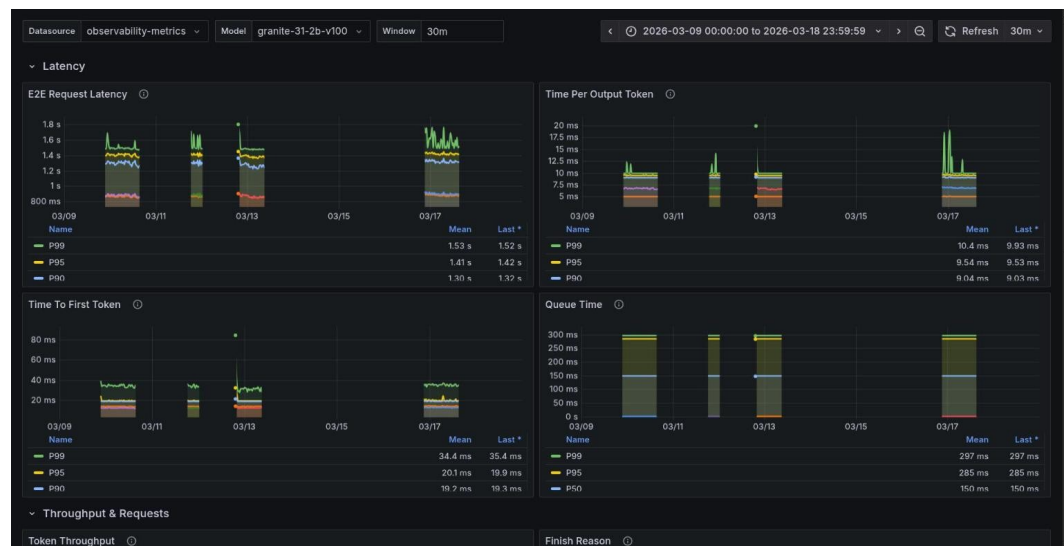
Console log view, backed by Loki



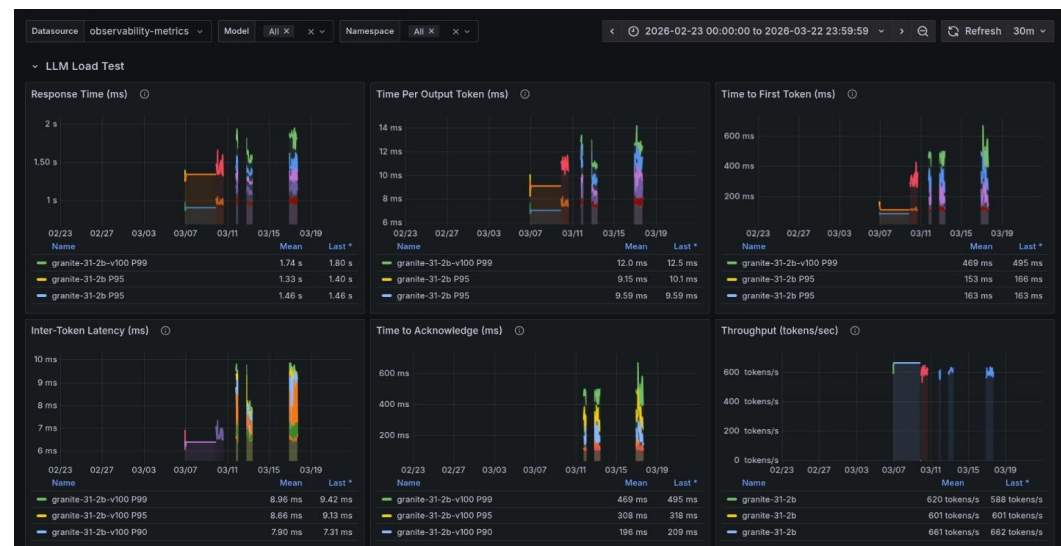
Terminal: oc logs, straight at a single pod



AI telemetry: what a served model looks like



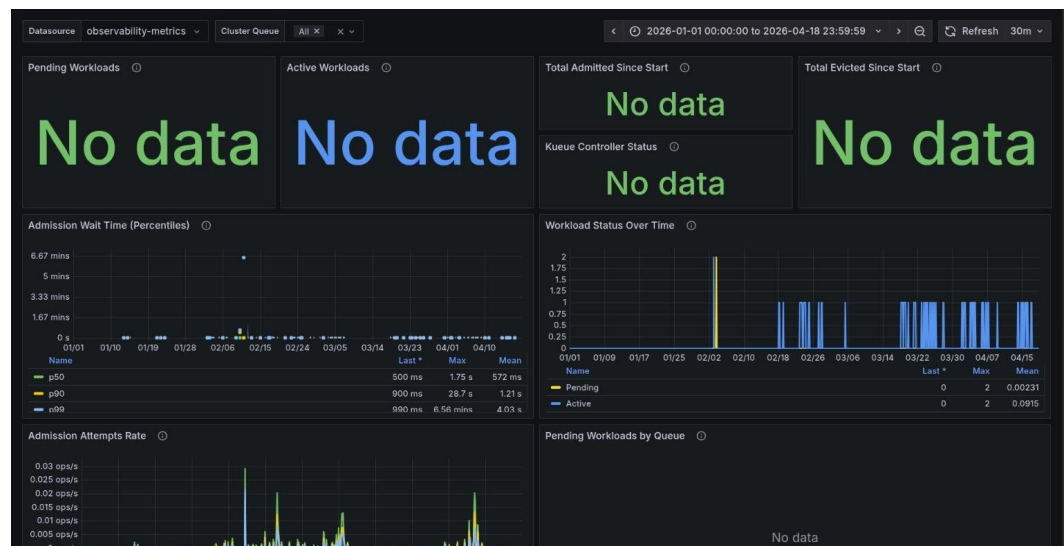
Grafana: time to first token, throughput, queue time



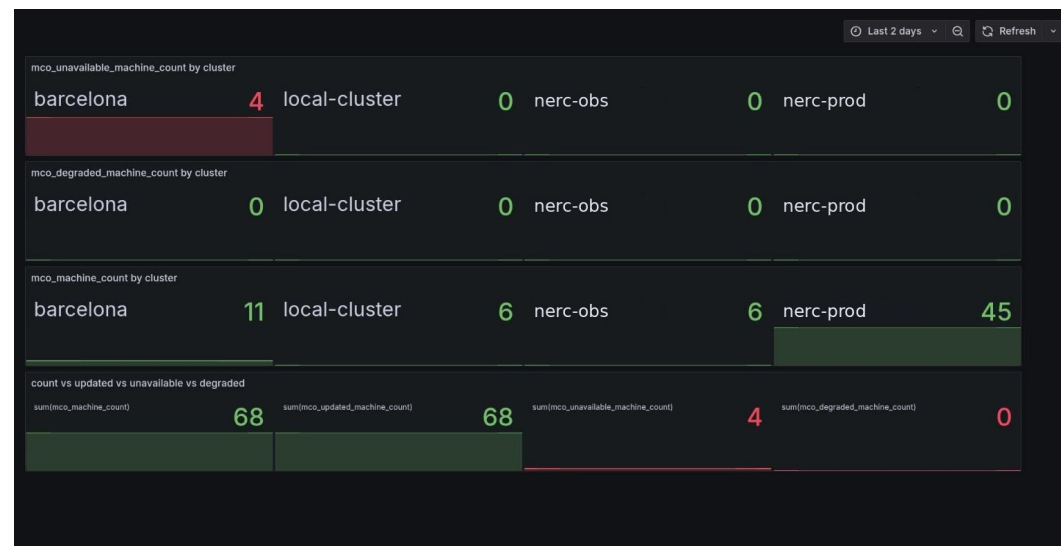
Grafana: the same model under sustained load



Scheduling and platform state



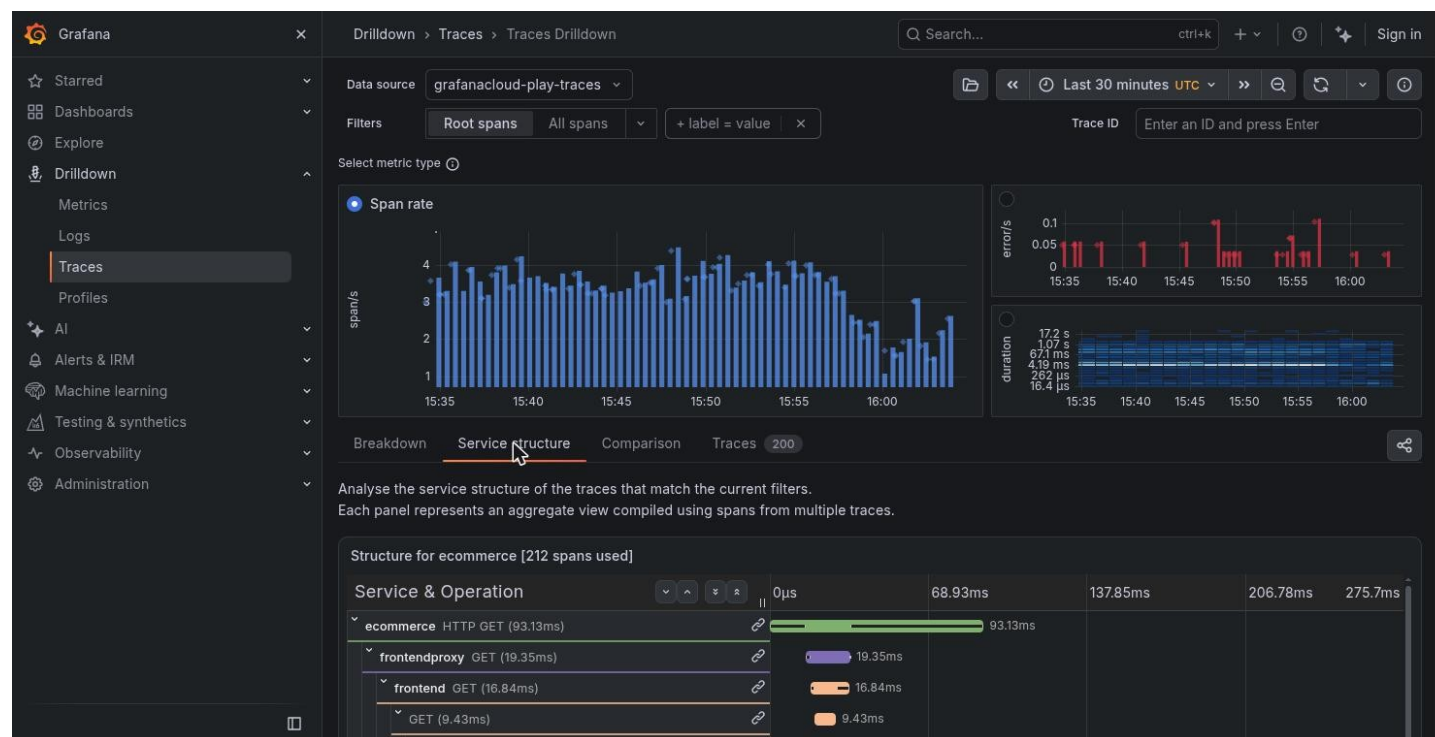
Kueue: admitted and active workloads



Machine config: what is blocked, and where



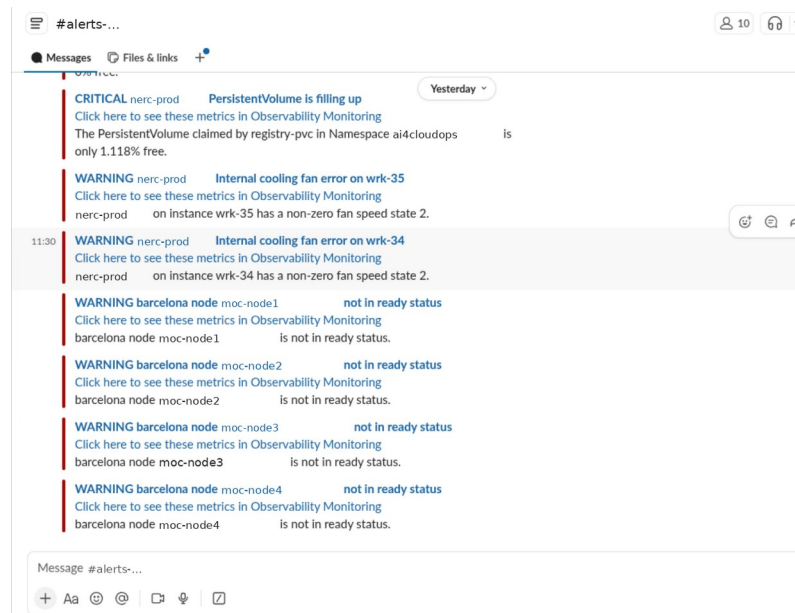
Traces: where the time goes



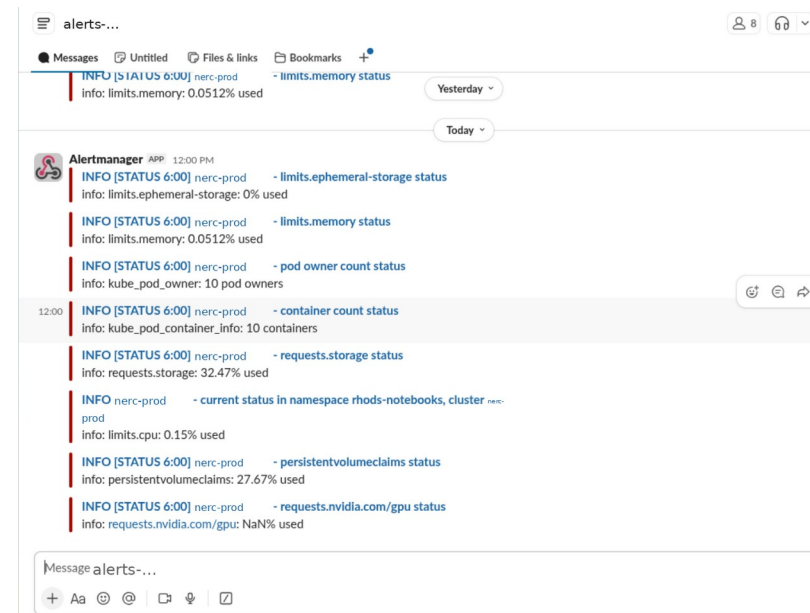
Service structure across 212 spans. From play.grafana.org, not this cluster



Alerts: where they land



Incidents: volumes filling up, nodes not ready



Scheduled status, nothing wrong

Slack is our choice, not a requirement. OpenShift ships Alertmanager receivers for PagerDuty, email, Microsoft Teams and Slack, plus a generic webhook for anything else a team already runs. Email needs an SMTP server of your own. What matters is that someone sees them.



Three layers of visibility, working together

Workload and managed cluster

A managed cluster is one of the OpenShift clusters MOC operates for you.

- OpenShift console and CLI
- Local workload metrics, logs and events
- Fast troubleshooting close to the application

RHOAI observability

RHOAI can give you a dedicated AI focused layer. Whether it's enabled for your project is a setup decision, just ask us.

- **Prometheus** collects and stores metrics
- **Alertmanager** routes alerts to the right people
- **Tempo** stores traces
- **OpenTelemetry Collector** receives and forwards telemetry from your code

Logging and Grafana are not part of this dedicated stack by default, but you can get both from the central layer below, or deploy them yourself.

Central multi-cluster observability

- **Prometheus** collects metrics per cluster, **Thanos** aggregates them across clusters and keeps long-term history
- **Loki** stores logs centrally
- **Grafana** provides the dashboards on top
- One combined view across multiple managed clusters

Key message Local and specialized observability complement the central MOC multi-cluster view. You're not choosing one over the other.



Each question maps to a different signal

What do you want to know?	Which signal answers it
Is my application healthy?	Metrics and dashboards
What caused the failure?	Logs and events
Where is the request delayed?	Traces
Which code consumes the most resources?	Continuous profiling
Do I need to know when this breaks?	Alerts
How efficiently is my model running?	AI and GPU metrics

Key message The question determines the signal, not the other way around.



Observability, inventory and billing answer different questions

Observability can support inventory and billing processes, but it is not the authoritative system for either.

Inventory gives you context

- What resources exist?
- Who owns them?
- What capacity is available?
- Which resources should be reporting?

Observability only sees resources that are discovered, connected and measurable. Anything unknown or unmonitored stays a blind spot, by definition.

Billing adds commercial rules

Dashboards can visualize usage, estimated cost and quotas, but that's not the same as an auditable billing process. The next slide shows why.

Key message Observability shows what is visible. Inventory defines what exists. Billing decides what should be charged.



From telemetry to invoice

Telemetry → Metering → Rating and Pricing → Billing → Invoice

- **Telemetry** records what was technically observed.
- **Metering** determines the consumed, billable units.
- **Rating and pricing** assign a monetary value.
- **Billing** applies contracts, credits and billing periods.
- **Invoice** produces the formal charge.

Each step adds rules the previous step doesn't know about. A dashboard sits at the first step only.

Key message A usage graph is not an invoice. It was never meant to be one.



Observability is nothing without a question.

Start with what you actually want to understand. Not with the metrics, and not with the dashboard.



What we can help you with

The main challenge is usually not getting access to data. It's deciding:

- Which metrics actually matter?
- How should the values be read?
- Which thresholds mean there's a real problem?
- Do you need a dashboard, an alert, a log query, or a trace?
- Is more instrumentation needed?

What you won't get automatically: not every possible metric is collected by default. Collection frequency, the number of distinct label values (label cardinality), and retention all affect system load and storage, so the defaults are deliberately conservative.

Where we can help

- Getting you set up with self-service access
- Looking at your goals with you and pointing out good practices
- Helping pick useful metrics and queries
- Building dashboards together with you
- Setting up dashboards and alerts for your specific case, if needed

Key message Just ask. We're not only here to give you tools and data, we can help you turn that into something useful for your workloads.



Thank you

Red Hat is the world's leading provider of enterprise open source software solutions, using a community-powered approach to deliver reliable and high-performing Linux, hybrid cloud, container, and Kubernetes technologies.



linkedin.com/company/red-hat



youtube.com/@redhat



facebook.com/redhatinc



x.com/RedHat

Glossary – Platform and environment

- **MOC 2.0** – Mass Open Cloud 2.0. The cloud platform this session is about, built on OpenShift.
- **OpenShift** – Red Hat's Kubernetes platform. It runs your containers and gives you the console and CLI you use.
- **RHOAI** – Red Hat OpenShift AI. The platform layer on top of OpenShift for training, serving and managing AI models.
- **Managed cluster** – one of the OpenShift clusters that the MOC team operates on your behalf. Your workloads run inside it.
- **Multi-cluster** – spanning several clusters at once. Central observability collects from all managed clusters into one view.
- **Pod** – the smallest deployable unit in Kubernetes: one or more containers that run together and share a network address.
- **Workbench** – in RHOAI, an interactive notebook environment for development and experimentation.
- **Inference service** – a deployed model that answers prediction requests over the network.
- **oc** – the OpenShift command line tool. Basically `kubectl` with OpenShift specific additions.



Glossary – Signals and concepts

- **Telemetry** – the umbrella term for all data a system emits about itself: metrics, logs, traces, events.
- **Metrics** – numeric measurements sampled over time, such as requests per second or memory in use.
- **Logs** – timestamped text records written by applications and platform components.
- **Traces** – records that follow one request across multiple services, showing where time was spent.
- **Events** – records of state changes in the platform, such as a pod restart or a failed image pull.
- **Continuous profiling** – ongoing, low overhead sampling of which code paths consume CPU or memory inside a running process.
- **Instrumentation** – the code that makes an application emit telemetry. Without it, some signals simply don't exist.
- **Alert** – a rule that watches a metric and notifies someone when a condition is met.
- **Threshold** – the value at which an alert fires. Choosing it well is the hard part.
- **Label cardinality** – the number of distinct values a metric label can take. High cardinality (say, one label value per user ID) multiplies storage and query cost fast.
- **Retention** – how long collected telemetry is kept before it gets deleted.
- **Blind spot** – something that exists but isn't being observed, so it can't show up in any dashboard or alert.



Glossary – AI-specific signals

- **Time to first token (TTFT)** – how long a model takes to produce its first output token after a request comes in. This is basically the user perceived responsiveness of a generative model.
- **Token throughput** – how many tokens the model produces per second once it has started.
- **Queue time** – how long a request waited before the model actually started processing it.
- **GPU utilization** – how busy the GPU is. Low utilization with a long queue usually points to a bottleneck somewhere else.



Glossary – Tools

- **Prometheus** – collects and stores metrics by scraping them from your workloads at regular intervals.
- **Thanos** – sits on top of Prometheus to aggregate metrics across clusters and store long-term history.
- **Loki** – stores and queries logs, built to work alongside Prometheus.
- **Tempo** – stores and queries traces.
- **Grafana** – builds dashboards and visualizations on top of metrics, logs and traces.
- **Perses** – a dashboard tool with dashboards defined as code, so they can be versioned like source.
- **Alertmanager** – receives alerts from Prometheus and decides who gets notified, how, and whether to group or silence them.
- **OpenTelemetry** – a vendor neutral standard for producing and transporting telemetry.
- **OpenTelemetry Collector** – the component that receives telemetry from your applications, processes it, and forwards it to backends such as Prometheus or Tempo.



Glossary – Commercial terms

- **Metering** – converting raw telemetry into countable, billable units.
- **Rating and pricing** – assigning a monetary value to metered units.
- **Billing** – applying contracts, credits and billing periods to priced usage.
- **Invoice** – the formal, auditable charge issued to a customer.
- **Quota** – a configured limit on how much of a resource a project may consume.



References

Verified as of 2026-09-08. Product documentation is version specific, check for a newer version before reusing this deck.

- [OpenTelemetry signals](#)
- [RHOAI 2.25 – Managing observability](#)
- [OpenShift 4.20 – Accessing metrics as a developer](#)
- [OpenShift 4.20 – Configuring alerts and notifications](#)

